

TP6 : Création d'une API REST/GraphQL

Test 1 : POST : Créer un utilisateur via REST

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/api/medecins`
- Method:** `POST`
- Body (JSON):**

```
{  "name": "Ali Ben Salah",  "email": "ali@example.com",  "password": "123456"}
```
- Status:** 201 Created
- Time:** 37 ms
- Size:** 355 B
- Response (JSON):**

```
{  "id": "8149f932-f479-4fbd-8bed-f5f0cba667d7",  "name": "Ali Ben Salah",  "email": "ali@example.com",  "password": "123456"}
```

Test 2 — POST : Créer un deuxième utilisateur via REST

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/api/medecins`
- Method:** `POST`
- Body (JSON):**

```
{  "name": "Koukou Lassoued",  "email": "koukou@example.com",  "password": "pass123"}
```
- Status:** 201 Created
- Time:** 8 ms
- Size:** 361 B
- Response (JSON):**

```
{  "id": "92fe8813-59f5-4fd8-9f07-e79827013b24",  "name": "Koukou Lassoued",  "email": "koukou@example.com",  "password": "pass123"}
```

Test 3 — GET : Récupérer tous les utilisateurs

HTTP

localhost:3000/api/medecins

GET

▼

http://localhost:5000/users

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

☒ none

☐ form-data

☐ x-www-form-urlencoded

☐ raw

☐ binary

This request does not have a body

Body

Cookies

Headers (7)

Test Results

Pretty

Raw

Preview

Visualize

JSON ▼

1

[

2

{

3

"id": "8149f932-f479-4fbd-8bed-f5f0cba667d7",

4

"name": "Ali Ben Salah",

5

"email": "ali@example.com",

6

"password": "123456"

7

},

8

{

9

"id": "92fe8813-59f5-4fd8-9f07-e79827013b24",

10

"name": "Koukou Lassoued",

11

"email": "koukou@example.com",

12

"password": "pass123"

13

}

14

]

Test 4 — GET : Récupérer un utilisateur par ID

POST localhost:3000/api/patients GET localhost:3000/api/medecins X PUT localhost:3000/api/patients GET localhost:3000/api/medecins + ...

localhost:3000/api/medecins

GET http://localhost:5000/users/92fe8813-59f5-4fd8-9f07-e79827013b24

Params Authorization Headers (6) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

This request does not have a body

Body Cookies Headers (7) Test Results Status: 200 OK Time: 4 ms Size: 356 B

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": "92fe8813-59f5-4fd8-9f07-e79827013b24",
3   "name": "Koukou Lassoued",
4   "email": "koukou@example.com",
5   "password": "pass123"
6 }
```

Test 5 — PUT : Modifier un utilisateur

POST localhost:3000/api/patients GET localhost:3000/api/medecins PUT localhost:3000/api/patients PUT localhost:3000/api/medecins + ...

localhost:3000/api/medecins

PUT http://localhost:5000/users/92fe8813-59f5-4fd8-9f07-e79827013b24

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

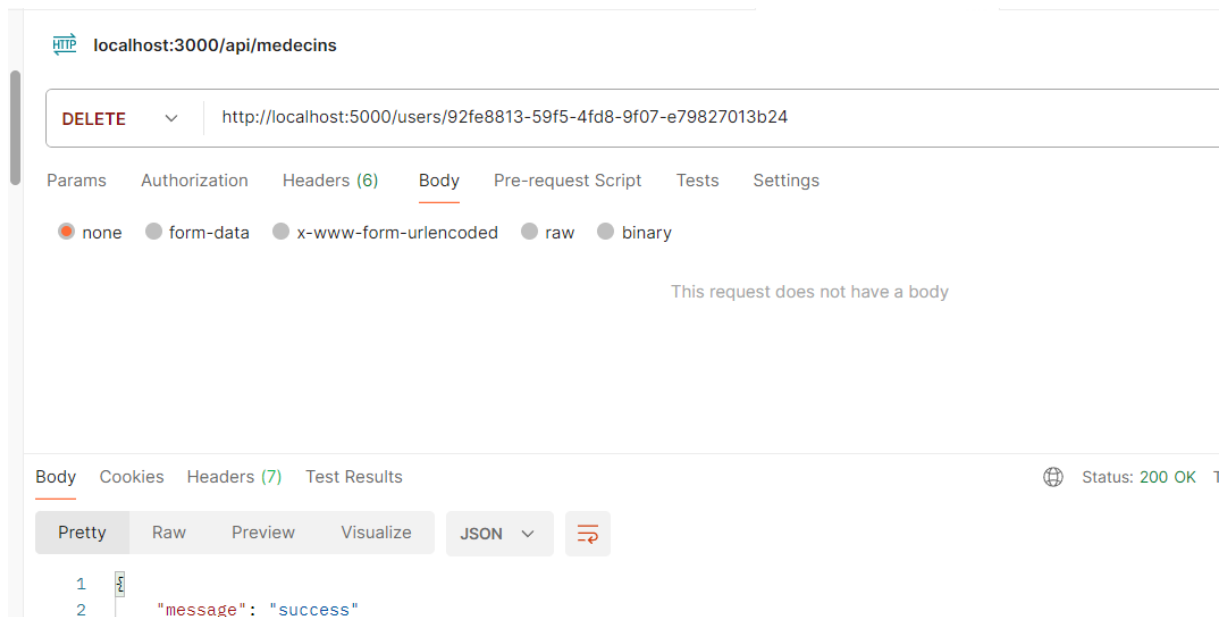
```
1 {
2   "name": "Ali Modifié",
3   "email": "ali.modifie@example.com",
4   "password": "newpass"
5 }
```

Body Cookies Headers (7) Test Results Status: 200 OK Time: 7 ms Size: 358 B

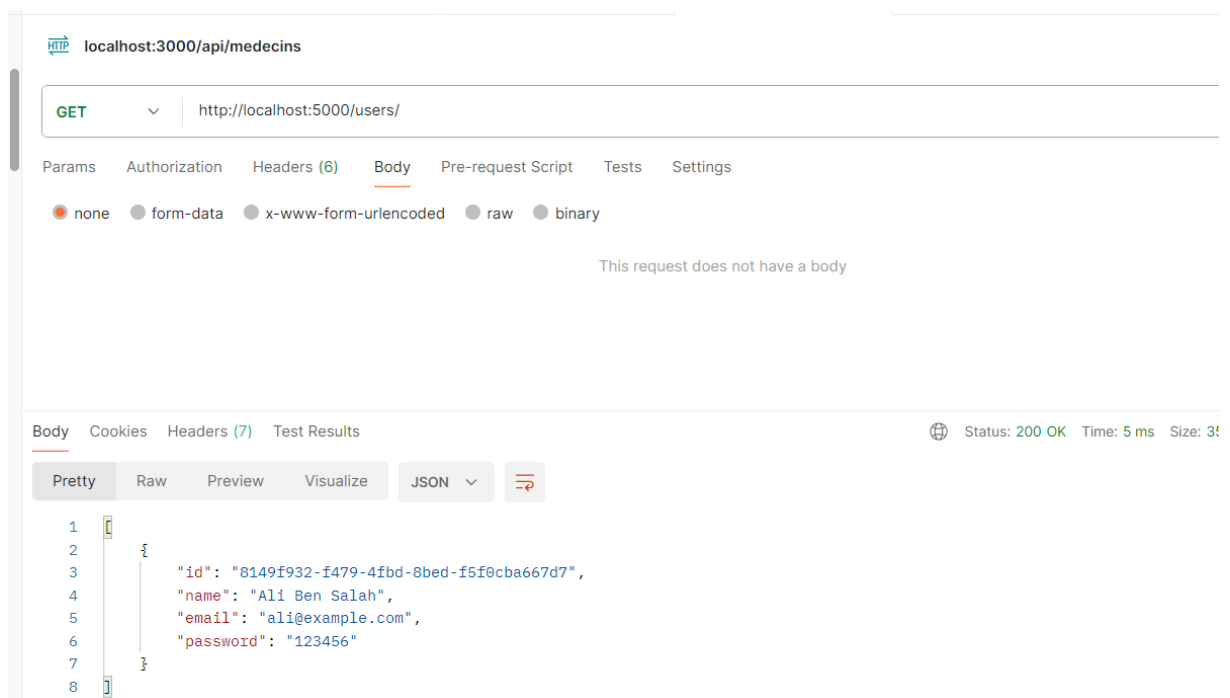
Pretty Raw Preview Visualize JSON

```
1 {
2   "id": "92fe8813-59f5-4fd8-9f07-e79827013b24",
3   "name": "Ali Modifié",
4   "email": "ali.modifie@example.com",
5   "password": "newpass"
6 }
```

Test 6 — DELETE : Supprimer un utilisateur



Vérification



ÉTAPE 11 — Tester GraphQL avec Postman

Test 1 — Ajouter un utilisateur via GraphQL

POST localhost:3000/api/patients GET localhost:3000/api/medecins PUT localhost:3000/api/patients POST localhost:3000/api/medecins + ...

localhost:3000/api/medecins Save

POST http://localhost:5000/graphql Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "query": "mutation { addUser(name: \"Amira\", email: \"amira@example.com\", password: \"abc123\") { id name email } }"
3 }
```

Body Cookies Headers (6) Test Results Status: 200 OK Time: 20 ms Size: 310 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "data": {
3     "addUser": {
4       "id": "0922e513-2d68-490e-b3df-5db01d9c1c04",
5       "name": "Amira",
6       "email": "amira@example.com"
7     }
8   }
9 }
```

Test 2 — Consulter tous les utilisateurs via GraphQL

localhost:3000/api/medecins

POST http://localhost:5000/graphql

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "query": "{ users { id name email } }"
3 }
```

Body Cookies Headers (6) Test Results Status: 200 OK

Pretty Raw Preview Visualize JSON

```
1 {
2   "data": {
3     "users": [
4       {
5         "id": "8149f932-f479-4fbd-8bed-f5f0cba667d7",
6         "name": "Ali Ben Salah",
7         "email": "ali@example.com"
8       },
9       {
10        "id": "0922e513-2d68-490e-b3df-5db01d9c1c04",
11        "name": "Amira",
12        "email": "amira@example.com"
13      }
14    ]
15  }
16 }
```

Test 3 — Vérifier via REST que l'utilisateur GraphQL existe

HTTP localhost:3000/api/medecins

GET http://localhost:5000/users/

Params Authorization Headers (6) **Body** Pre-request Script Tests Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary

This request does not have a body

Body Cookies Headers (7) Test Results 🌐 Status: 200 OK Time:

Pretty Raw Preview Visualize JSON

```
1 {
2   {
3     "id": "8149f932-f479-4fbd-8bed-f5f0cba667d7",
4     "name": "Ali Ben Salah",
5     "email": "ali@example.com",
6     "password": "123456"
7   },
8   {
9     "id": "0922e513-2d68-490e-b3df-5db01d9c1c04",
10    "name": "Amira",
11    "email": "amira@example.com",
12    "password": "abc123"
13  }
14 }
```

Test 4 — Mise à jour via GraphQL

HTTP http://localhost:5000/graphql 📄 Save

POST http://localhost:5000/graphql Send

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cooki

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary JSON Beautif

```
1 {
2   query: "mutation { updateUser(id: \"0922e513-2d68-490e-b3df-5db01d9c1c04\", name: \"Amira K.\", email: \"amira.k@example.com\",
3     password: \"xyz789\") { id name email } }"
```

Body Cookies Headers (6) Test Results 🌐 Status: 200 OK Time: 7 ms Size: 318 B Save Respons

Pretty Raw Preview Visualize JSON

```
1 {
2   "data": {
3     "updateUser": {
4       "id": "0922e513-2d68-490e-b3df-5db01d9c1c04",
5       "name": "Amira K.",
6       "email": "amira.k@example.com"
7     }
8   }
9 }
```

Test 5 — Suppression via GraphQL

HTTP <http://localhost:5000/graphql>

POST [▼](#) <http://localhost:5000/graphql>

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary JSON [▼](#)

```
1 {
2   "query": "mutation { deleteUser(id: \"0922e513-2d68-490e-b3df-5db01d9c1c04\") }"
3 }
```

Body Cookies Headers (6) Test Results 🌐 Status: 200 OK Time: 5 ms Size: 229 B [Sa](#)

Pretty Raw Preview Visualize JSON [▼](#) [🔗](#)

```
1 {
2   "data": {
3     "deleteUser": true
4   }
5 }
```

✓ Vérification

HTTP <http://localhost:5000/graphql>

GET [▼](#) <http://localhost:5000/users/>

Params Authorization Headers (6) **Body** Pre-request Script Tests Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary

This request does not have a body

Body Cookies Headers (7) Test Results 🌐 Status: 200 OK Time: 5 ms Size: 352 B [Sa](#)

Pretty Raw Preview Visualize JSON [▼](#) [🔗](#)

```
1 {
2   "id": "8149f932-f479-4fbd-8bed-f5f0c6a667d7",
3   "name": "Ali Ben Salah",
4   "email": "ali@example.com",
5   "password": "123456"
6 }
7
8 }
```

- ⇒ Ce TP démontre l'interopérabilité entre REST et GraphQL sur la même base de données. RxDB joue le rôle de couche de persistance partagée — une donnée créée via REST est immédiatement accessible via GraphQL et vice versa. L'ajout du type Device illustre la relation de composition : un Device ne peut pas exister sans User.